

September 27, 2026

Photography Equipment Rental

Introduction

Every successful business relies on accurate, well-organized information to run efficiently. A photography equipment rental company needs to track several categories of business data, including *equipment availability, customer contact information, employee records, rental transactions and return dates, payments, and equipment repairs*. As the company's rental activity increases, disorganized records can lead to inaccurate information, double bookings, missed return dates, and payment errors. To reduce these risks, the company should implement an SQL-based database management system to centralize its rental data, maintain organized business records, and generate accurate operational reports.

For this project, I created and implemented a database for a single-location rental business. The completed database, `photography_rental`, includes nine connected tables developed in MySQL Workbench, filled with sample records, and evaluated with ten queries that address routine business needs. This report explains the business requirements, table structure, SQL implementation, development challenges, and lessons learned.

Business Rules and Table Design

I gathered the requirements by following a rental from checkout to return and deciding what the shop needed to record. The database tracks customers, employees, equipment, rentals, payments, and repairs.

Business Rules

These eight rules explain how records connect and whether each connection is required:

1. **Customer and Rental:** A customer can have zero or more rentals. Each rental belongs to exactly one customer.
2. **Employee and Rental:** An employee can check out zero or more rentals. Exactly one employee checks out each rental.
3. **Employee and Rental Item:** An employee can check in zero or more items. Each item has at most one check-in employee, recorded when it is returned.
4. **Rental and Rental Item:** Each rental includes at least one item. Each item belongs to exactly one rental.
5. **Rental Item and Equipment Unit:** Each rental item refers to one equipment unit. A unit can be rented many times over time.
6. **Equipment Model and Equipment Unit:** Each model has at least one unit. Each unit belongs to exactly one model.
7. **Rental and Payment:** Each rental has at least one payment collected at checkout. Each payment belongs to exactly one rental.
8. **Repair Order and Equipment Unit:** Each repair order covers at least one unit. A unit can have zero or more repair orders, with a separate cost and reason for each repair.

Table Design

The database has nine connected tables, each storing information about one part of the business. Primary keys identify individual records, while foreign keys connect related records across tables. For example, the customer ID in Rental

connects a rental to its customer. This link shows which customers made the rental without repeating their name and contact details in the Rental table.

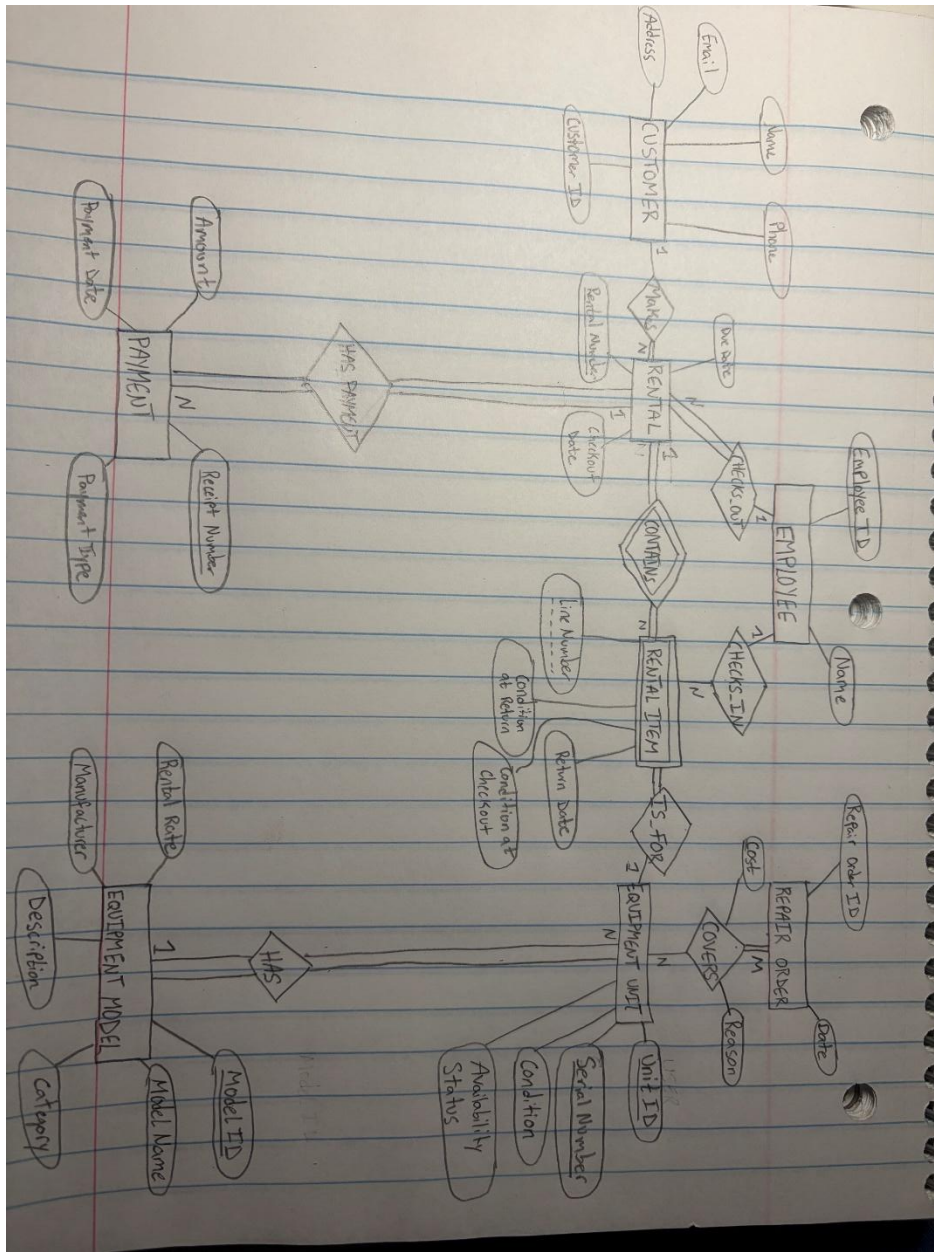
Equipment Model and *Equipment Unit* are separate because they store different details. A model stores shared information, such as the manufacturer, model name, and daily rental rate. A unit stores details about one physical item, including its serial number, condition, and availability. Keeping them separate avoids repeating model information while allowing each physical item to be tracked individually.

Rental stores information about the whole rental, such as its checkout and due dates. *Rental Item* stores each item's return date, return condition, and check-in employee because items may be returned separately. The rental number and line number together identify each rental item.

The *repair_order_unit* table connects repair orders to equipment units. This allows one repair order to cover several units and one unit to appear on several repair orders. It stores the cost and reason for each unit's repair and uses the repair order ID and unit ID together as its primary key.

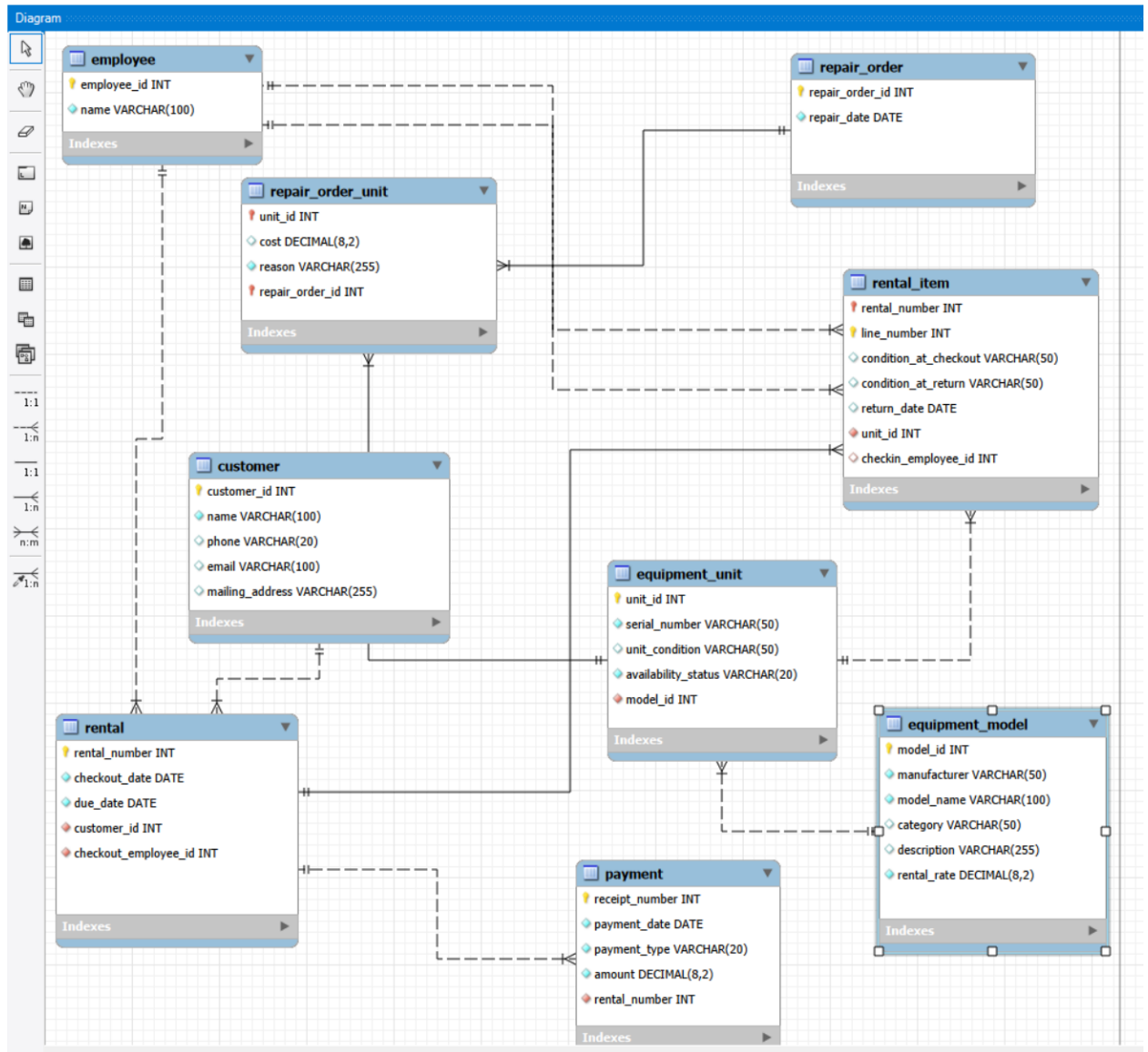
The hand-drawn ER diagram

This handwritten ER diagram shows what the database tracks and how the parts connect. It also shows which connections are required or optional.



MySQL Workbench diagram

This diagram shows the nine tables, the information they store, and the keys that connect them.



Building and Testing the Database

Creating the Tables

Once my model was finished, I used the Forward Engineer tool under the Database menu to turn my design into SQL. At first, I wasn't sure whether I should

keep clicking Next, but the tool walked me through each step. On the review screen, Workbench showed me the CREATE SCHEMA and CREATE TABLE statements it had written from my diagram. I saved that script as photography_rental_schema.sql before running it so that I would have a copy for my project. Along the way, I also learned that the model and the SQL editor are two different places in Workbench. When the tool finished, I opened my local connection and ran SHOW TABLES to make sure all nine tables were there.

Adding Sample Data

I used my favorite celebrities such as Usher and BTS as customers and NBA players such as LeBron James as employees. The sample phone numbers were based on Spotify monthly listener counts, with zeros added at the end to make each number ten digits; these numbers were used only as test data. I added records in an order that followed the foreign key connections. *Customers, employees, equipment models, and repair orders* came first, followed by *equipment units and rentals*. Rental items, payments, and repair-order units came last so their foreign keys could connect to existing records. Running the script a second time caused duplicate records and changed the automatic IDs. A reset step fixed this by clearing the tables and restarting the automatic IDs at 1 before adding the sample data again.

Testing with Queries

To test the database, I acted out a customer interaction and asked ten questions a rental shop might need to answer. For example, if a customer wanted to rent a camera, the first query showed what was available. Other questions included what equipment was still out, which items came back late, and which customers had never rented. I added comments to explain each query and a short answer at the end to make the results easy to check.

The results matched the sample data. Six units were available, the same Sony A7 IV came back late twice, and The Notorious B.I.G. still had three items out. The last query changed the repair cost from NULL (meaning no cost had been entered yet) to \$250. I placed it last because it changes the data, allowing the other queries to use the original records first.

Problems and Solutions

A few problems came up while building and testing the database. Most involved missing settings, incorrect dates, or running the same script twice. Fixing these problems helped me understand how the tables and records worked together.

The biggest issue happened when the sample data script ran a second time. It created duplicate records and continued the automatic IDs instead of starting over. Adding a reset step fixed this by clearing the tables and restarting the IDs before reloading the records.

Some table settings also needed changes. The email column was missing its UNIQUE setting, which prevents duplicate email addresses. In *repair_order_unit*, only one column was marked as the primary key. I corrected this by selecting both the repair order ID and unit ID so they could identify each record together.

Dates caused another problem. Some were entered as 03/17/2026, so I changed them to the YYYY-MM-DD format, such as 2026-03-17. Some repair dates also came before the damaged equipment was returned. I adjusted those dates so the events followed the correct order. The apostrophe in Shaquille O'Neal's name also needed attention. Using two apostrophes in the SQL text let me enter the name correctly.

These mistakes taught me to test each step, read error messages carefully, and rerun the scripts. The script that worked the first time didn't always work again without changes.

Conclusion

At the start of this project, I proposed that an SQL database could help a photography rental shop keep its data in one place, organize its records, and generate accurate reports. The project involved translating business requirements into eight rules and nine tables, building and testing the database with ten queries, and fixing errors in the design and SQL scripts. The queries returned the expected results from the sample data, demonstrating the database's ability to track equipment, rentals, payments, and repairs.

The project taught me to plan the design before building, add records in the correct order, and test scripts multiple times. The database currently supports one location and does not handle future reservations. Future improvements could include a reservation system to prevent double bookings, automatic availability updates, and a PHP webpage for employees. These additions would make the database more useful for managing the shop's daily work.